AIRPORT MANAGEMENT APPLICATION

ICS1261 - Software Development and Practice using Python

FIRST REVIEW REPORT

Submitted By

1. Santhosh M
2. Jishnu P L
3. Sri Vikash N S

Department of Computer Science and Engineering
Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University)
Kalavakkam – 603110
February 2025

Subject in-charge

1. **Abstract**

The Efficient Airport Operations and Management System is a desktop-based software application developed using Python Tkinter for the front end and CSV files as the database. The system aims to manage airport operations including airline management, ticket booking, air traffic control, and check-in processes through secure module-based access using passkeys. The project solves inefficiencies in flight scheduling, passenger processing, crew management, and runway allocation by providing a centralized, modular platform that ensures smooth airport operations and controlled access to different departments.

2. **Introduction**

Background of the Problem

Modern airports handle multiple airlines, flights, crew members, passengers, and ground operations simultaneously. Managing these manually or through disconnected systems leads to delays, resource conflicts, and inefficiencies.

Why the System is Needed

- To centralize airport operations
- To reduce human errors
- To manage crew, flights, gates, and passengers efficiently
- To improve passenger experience
- To handle disruptions effectively

Overview of System Functionality

The system consists of four main modules:

- Manage Airlines
- Ticket Counter
- ATC (Air Traffic Control)
- Check-in

Each module is secured using a passkey and operates within a single Tkinter window.

3. Problem Statement

Problem Definition

Airports face challenges in managing flights, crew allocation, ticket booking, runway scheduling, and passenger processing efficiently.

Objectives of the System

- To design a modular airport management system
- To implement secure access using passkeys
- To manage airline, crew, and aircraft data
- To handle passenger booking and baggage constraints
- To manage runway and gate allocation

Expected Output

- Organized flight schedules
- Efficient seat allocation
- Controlled runway usage

- Validated baggage and check-in processes
- Reduced operational conflicts

4. Exploration of Problem Statement

Scope

- Airline data management
- Ticket booking and seat allocation
- Runway and gate scheduling
- Passenger check-in validation

Limitations

- Uses CSV instead of real-time database
- Desktop-based only
- No real-time weather integration
- No live airline API integration

Input Analysis

Inputs include:

- Flight details
- Passenger information
- Crew data
- Runway availability
- Baggage weight

Output Analysis

Outputs include:

- Updated flight schedule
- Seat confirmation
-
- Runway allocation status
- Boarding validation

Sample Use Cases

- ATC assigns runway for a flight
- Ticket counter books a seat for a passenger
- Airline admin updates aircraft details
- Check-in validates baggage weight

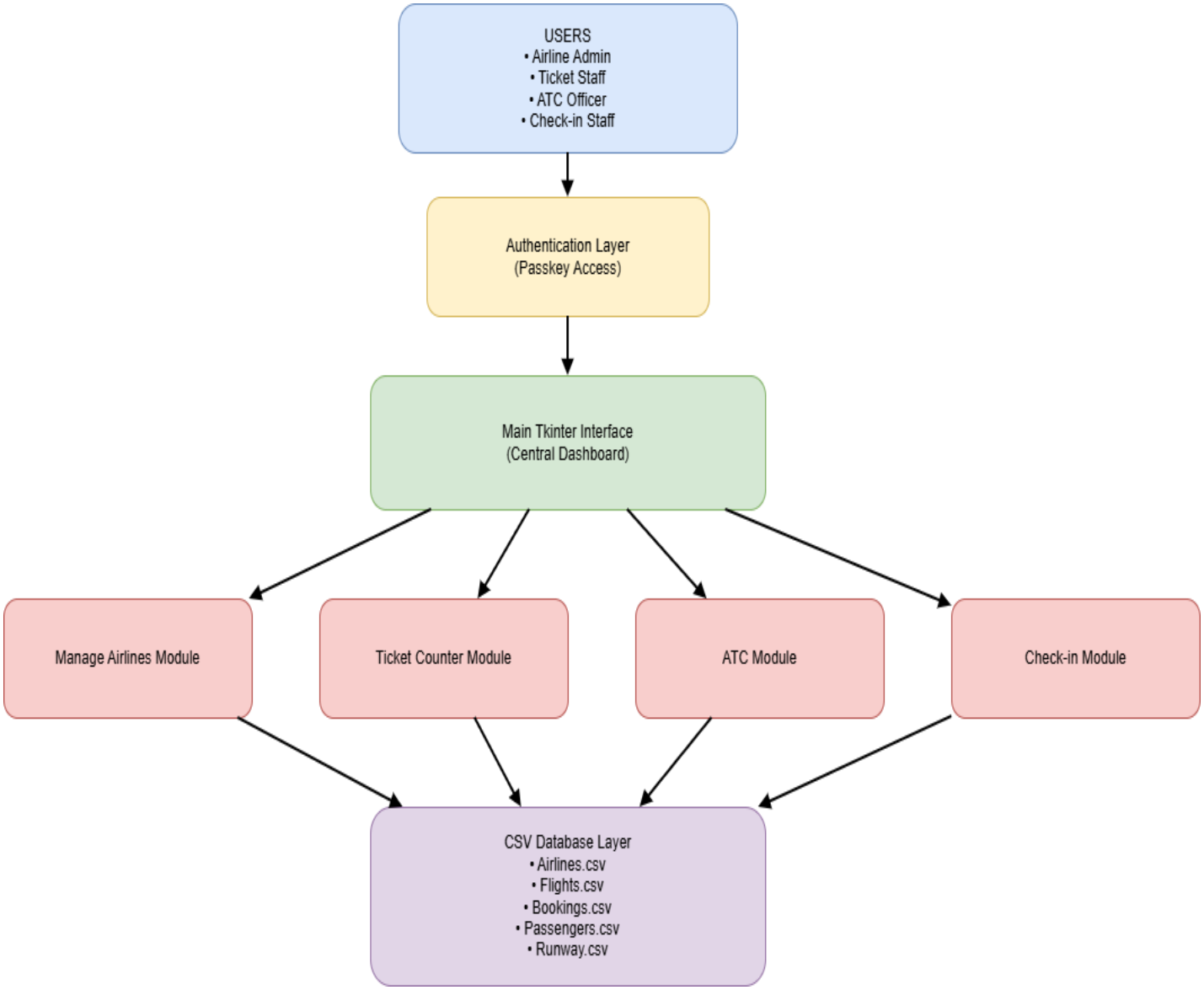
5. Literature review and Limitations

- Karahasanovic et al., "Can holistic optimization improve airport air traffic management performance? - Developing and evaluating an optimization algorithm," in *Proc. IEEE Int. Conf. Digit. Aviation Technol.*, 2019, doi: 10.1109/DAVT.2019.8953390.

- H. Danah et al., "Intelligent airport management system," in Proc. IEEE Int. Conf. Intell. Syst. Comput., 2023, doi: 10.1109/icisc.2023.10085083.

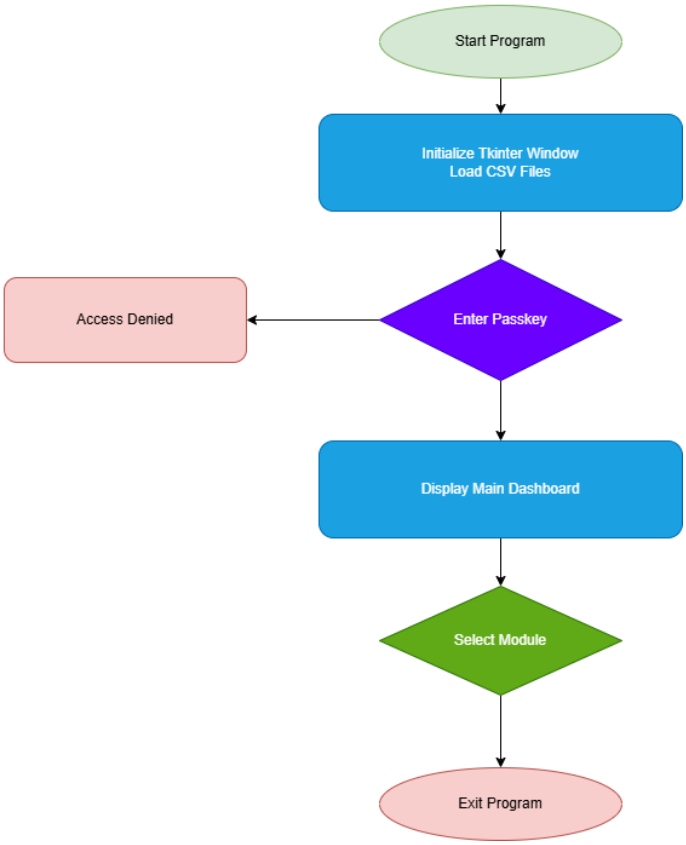
- M. A. Bouyakoub et al., "Intelligent Baggage Management in Airports: A Cognitive IoT Approach," in *Proc. IEEE Int. Conf. Innov. Intell. Syst. Appl.*, 2023, doi: 10.1109/inista59066.2023.10395201.

6. Block Diagram with Explanation

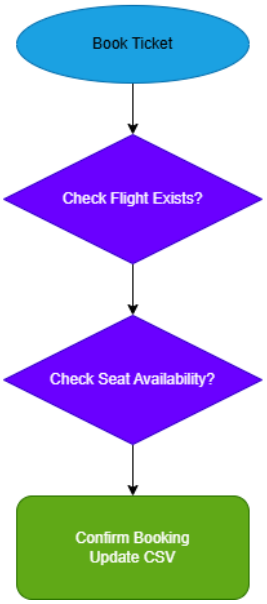


7. Functions Flowchart

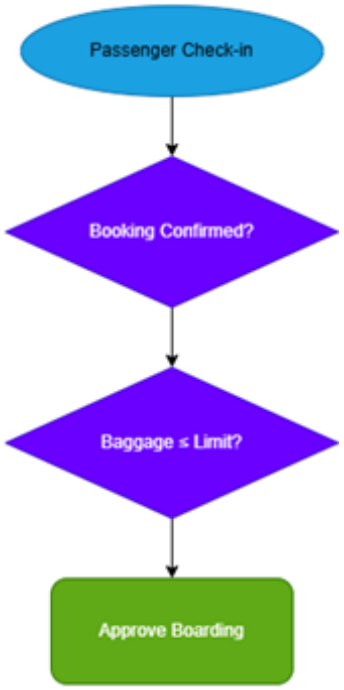
This flowchart illustrates the overall control flow of the application, from system initialization and authentication to module selection and program termination.



This flowchart illustrates the logical sequence involved in validating flight availability, checking seat status, and confirming passenger booking.



This flowchart demonstrates the validation process for confirmed bookings and baggage weight before approving boarding.



This flowchart represents the decision process for verifying runway availability and assigning slots without scheduling conflicts.

8. Modules with Constraints

1. Manage Airlines Module

Purpose

Manage airline, aircraft, and crew information.

Inputs

- Airline ID
- Aircraft data
- Crew details

Outputs

- Updated airline records
- Crew assignments

Constraints

- Unique airline ID
- Crew must belong to airline
- Aircraft type must match airline

2. Ticket Counter Module

Purpose

Handle booking and seat allocation.

Inputs

- Passenger details
- Seat number
- Flight ID

Outputs

- Seat confirmation
- Boarding details

Constraints

- Seat cannot be duplicated
- Passenger class must match available seats
- Flight must exist

3. ATC Module

Purpose

Manage gates, runways, and flight schedules.

Inputs

- Flight ID
- Runway availability
- Gate details

Outputs

- Assigned runway
- Updated flight schedule

Constraints

- One runway per time slot
- No overlapping gate assignment
- Flight must be active

4. Check-in Module

Purpose

Validate passenger boarding and baggage.

Inputs

- Passenger ID
- Baggage weight

Outputs

- Boarding approval
- Baggage validation status

Constraints

- Baggage weight ≤ limit
- Passenger booking must be confirmed

9. Timeline

Week	**Task**
Week 1	Requirement analysis & system design
Week 2	UI design using Tkinter
Week 3	Implement authentication & routing
Week 4	Develop Airline module
Week 5	Develop Ticket module
Week 6	Develop ATC module
Week 7	Develop Check-in module
Week 8	CSV integration & validation
Week 9	Testing & debugging
Week 10	Documentation & presentation preparation